NUMPY — FINAL COMPLETE NOTES

Data Analysis + Data Science | Theory + Practical Exercises

PART 1 — NUMPY BASICS

1. What is NumPy?

Definition: NumPy (Numerical Python) is a Python library used for fast numerical calculations, arrays, matrices, statistics and mathematical operations.

Uses:

- Numerical data analysis
- Data cleaning and preprocessing
- Statistical calculations
- Matrix operations in Machine Learning
- Normalization and feature scaling

Import NumPy:

```
import numpy as np
```

`np` is the standard alias for NumPy.

2. NumPy Array (ndarray)

Definition: A NumPy array stores elements in one or more dimensions and supports efficient mathematical operations.

```
import numpy as np

a = np.array([12, 13, 14, 15, 16])
print(a)
print(type(a))
```

Output:

```
[12 13 14 15 16]
<class 'numpy.ndarray'>
```

Array properties

```
print(a.ndim)
print(a.shape)
print(a.size)
print(a.dtype)
print(a.itemsize)
print(a.nbytes)
```

Meaning:

- `ndim` — number of dimensions.
- `shape` — size along each dimension.
- `size` — total number of elements.
- `dtype` — data type of elements.
- `itemsize` — bytes used by one element.
- `nbytes` — total bytes used by array elements.

Output for `a`:

```
1
(5,)
5
int64
8
40
```

Note: `dtype`, `itemsize` and output sizes can differ by platform or chosen data type.

Tricky point: These are attributes, not methods. Write `a.shape`, not `a.shape()`.

---

3. Array Creation Methods

`np.array()`

Creates an array from a list or another sequence.

```
a = np.array([1, 2, 3])
print(a)
```

Output:

```
[1 2 3]
```

`np.arange()`

Creates values at regular intervals. The stop value is excluded.

```
print(np.arange(7, 101, 7))
```

Output:

```
[ 7 14 21 28 35 42 49 56 63 70 77 84 91 98]
```

`np.zeros()`, `np.ones()` and `np.full()`

```
print(np.zeros((2, 3), dtype=int))
print(np.ones((2, 3), dtype=int))
print(np.full((2, 2), 9))
```

Output:

```
[[0 0 0]
 [0 0 0]]

[[1 1 1]
 [1 1 1]]

[[9 9]
 [9 9]]
```

Use: `np.full((4, 4), 9)` creates a 4×4 array filled with 9, without a loop.

`np.eye()`

Creates an identity matrix.

```
print(np.eye(3, dtype=int))
```

Output:

```
[[1 0 0]
 [0 1 0]
 [0 0 1]]
```

`np.linspace()`

Creates a specified number of equally spaced values.

```
print(np.linspace(0, 10, 5))
```

Output:

```
[ 0.   2.5  5.   7.5 10. ]
```

Difference:

- `arange(start, stop, step)` specifies the step size.
- `linspace(start, stop, num)` specifies the number of values.

Random arrays

```
rng = np.random.default_rng(42)

print(rng.integers(1, 10, size=5))
print(rng.random((2, 3)))
```

`integers(1, 10)` can generate 1 through 9; 10 is excluded.

PART 2 — OPERATIONS AND FILTERING

4. Arithmetic Operations

Definition: NumPy performs element-wise operations between compatible arrays.

```
a = np.array([12, 13, 14, 15, 16])
b = np.array([10, 12, 10, 95, 17])

print(a + b)
print(a - b)
print(a * b)
print(a / b)
print(a // b)
print(a % b)
print(a ** 2)
```

Output:

```
[ 22  25  24 110  33]
[ 2  1  4 -80 -1]
[ 120  156  140 1425  272]
[1.2        1.08333333 1.4        0.15789474 0.94117647]
[1 1 1 0 0]
[2 1 4 15 16]
[144 169 196 225 256]
```

Meanings:

- `+` addition
- `-` subtraction
- `*` element-wise multiplication
- `/` division
- `//` floor division
- `%` remainder (modulus)
- `**` power

Tricky point: `a * b` is element-wise multiplication, not matrix multiplication. Use `a @ b` for matrix multiplication when shapes are compatible.

5. Boolean Conditions and Filtering

Definition: Boolean indexing selects elements that satisfy a condition.

```
a = np.array([12, 13, 14, 15, 16])

print(a > 15)
print(a[a > 15])
```

Output:

```
[False False False False  True]
[16]
```

The first statement returns a Boolean mask. The second returns only matching elements.

Multiple conditions

```
print(a[(a > 12) & (a < 16)])
print(a[(a < 13) | (a > 15)])
```

Output:

```
[13 14 15]
[12 16]
```

Remember:

- `&` = AND
- `|` = OR
- `~` = NOT

Put each condition inside parentheses.

6. `np.where()`

Definition: `np.where(condition, value_if_true, value_if_false)` chooses a value according to a condition.

```
a = np.array([12, 13, 14, 15, 16])

ans = np.where(a % 2 == 0, "Yes", "No")
print(ans)
```

Output:

```
['Yes' 'No' 'Yes' 'No' 'Yes']
```

Keep even numbers and replace odd numbers

```
ans = np.where(a % 2 == 0, a, -1)
print(ans)
```

Output:

```
[12 -1 14 -1 16]
```

Tricky point: In `np.where(a % 2 == 0, a, _)`, `_` is not a special placeholder for a missing value. Unless `_` has already been defined, this raises a `NameError`. Use an actual value, such as `-1`, `0` or `np.nan` as appropriate.

If you mix numbers and strings, NumPy may convert the resulting array to a string dtype.

7. Boolean Indexing for Updating Values

Definition: A Boolean mask can also be used to modify matching elements.

```
arr = np.array([1, 3, 5, 6, 8, 9, 10, 12])

arr[arr % 3 == 0] = -1

print(arr)
```

Output:

```
[ 1 -1  5 -1  8 -1 10 -1]
```

All elements divisible by 3 are replaced by -1.

PART 3 — DIMENSIONS AND INDEXING

8. 1D and 2D Arrays

```
mat = np.array([[1, 2, 4],
                [3, 4, 5],
                [7, 8, 9]])

print(mat.shape)
print(mat.ndim)
print(mat.size)
```

Output:

```
(3, 3)
2
9
```

Difference between list and array:

```
l = [[1, 2, 4], [3, 4, 5], [7, 8, 9]]
mat = np.array(l)
```

`l` is a nested Python list. `mat` is a NumPy array with array properties and vectorized operations.

9. Indexing and Slicing

Definition: Indexing selects individual elements; slicing selects a range.

```
a = np.array([10, 20, 30, 40, 50])

print(a[0])
print(a[-1])
print(a[1:4])
print(a[::2])
print(a[::-1])
```

Output:

```
10
50
[20 30 40]
[10 30 50]
[50 40 30 20 10]
```

Slicing syntax: `array[start:stop:step]`

The stop index is excluded.

Tricky point: `a[len(a)-1:-1:-1]` returns an empty array because `-1` as the stop is interpreted relative to the array length and is excluded. Use `a[::-1]` for a simple reverse.

2D indexing

```
m = np.array([[1, 2, 3],
              [4, 5, 6]])

print(m[0, 1])
print(m[1, 2])
print(m[0, :])
print(m[:, 1])
```

Output:

```
2
6
[1 2 3]
[2 5]
```

- `m[row, column]` selects one element.
- `m[0, :]` selects the first row.
- `m[:, 1]` selects the second column.

10. Extracting the Four Corners of a Matrix

Question: Create a 6×6 array and extract the four corner elements.

```
arr = np.arange(1, 37).reshape(6, 6)

print(arr)
print(arr[0, 0], arr[0, -1],
      arr[-1, 0], arr[-1, -1])
```

Output:

```
[[ 1  2  3  4  5  6]
 [ 7  8  9 10 11 12]
 [13 14 15 16 17 18]
 [19 20 21 22 23 24]
 [25 26 27 28 29 30]
 [31 32 33 34 35 36]]

1 6 31 36
```

Important: `arr[0:2, 0:2]` extracts a 2×2 block, not just the four corners.

11. Extract Every Third Element

Question: Extract every third element starting from index 1.

```
arr = np.arange(1, 13)

print(arr[1::3])
```

Output:

```
[ 2  5  8 11]
```

Reason: Index 1 is the starting position, and step 3 means skip two elements between selections.

12. Reverse the Rows of a Matrix

```
arr = np.arange(1, 26).reshape(5, 5)

print(arr[::-1])
```

Output:

```
[[21 22 23 24 25]
 [16 17 18 19 20]
 [11 12 13 14 15]
 [ 6  7  8  9 10]
 [ 1  2  3  4  5]]
```

`arr[::-1]` reverses the order of rows. It does not reverse the elements within each row.

To reverse both rows and columns, use `arr[::-1, ::-1]`.

PART 4 — RESHAPE, TRANSPOSE AND SPLITTING

13. `reshape()`

Definition: Changes the shape of an array without changing its number of elements.

```
arr = np.arange(1, 17)
matrix = arr.reshape(4, 4)

print(matrix)
print(matrix.shape)
```

Output:

```
[[ 1  2  3  4]
 [ 5  6  7  8]
 [ 9 10 11 12]
 [13 14 15 16]]

(4, 4)
```

Rule: Total elements must remain equal.

For example, 12 elements can be reshaped to `(3, 4)` or `(2, 6)`, but not `(5, 3)`.

Using `-1`

```
print(np.arange(12).reshape(3, -1).shape)
```

Output:

```
(3, 4)
```

NumPy calculates the unknown dimension automatically.

14. Transpose

Definition: Transpose swaps axes. In a 2D matrix, rows become columns and columns become rows.

```
arr = np.arange(1, 10).reshape(3, 3)

print(arr)
print(arr.T)
print(np.transpose(arr))
```

Output:

```
[[1 2 3]
 [4 5 6]
 [7 8 9]]

[[1 4 7]
 [2 5 8]
 [3 6 9]]

[[1 4 7]
 [2 5 8]
 [3 6 9]]
```

Verify transpose of transpose

```
check = arr.T
again = check.T

print(np.array_equal(arr, again))
```

Output:

```
True
```

`np.array_equal()` checks whether two arrays have equal shapes and equal elements.

Tricky distinction:

- `arr.T` transposes axes.
- `arr.flatten(order="F")` reads values column by column into a 1D array.
- These are different operations.

For an N-dimensional array, `.T` reverses all axes; `np.transpose(arr, axes=...)` allows a custom axis order.

15. `flatten()` and `ravel()`

Definition: Both convert a multidimensional array into one dimension.

```
arr = np.array([[1, 2], [3, 4]])

print(arr.flatten())
print(arr.ravel())
print(arr.flatten(order="F"))
```

Output:

```
[1 2 3 4]
[1 2 3 4]
[1 3 2 4]
```

- `flatten()` always returns a copy.
- `ravel()` returns a view when possible, otherwise a copy.
- `order="F"` means column-major element order.

16. `np.concatenate()`

Definition: Joins two or more arrays along an existing axis.

Join 1D arrays

```
x = np.array([12, 13, 14, 15, 16])
y = np.array([10, 12, 10, 95, 17])

res = np.concatenate((x, y))

print(res)
```

Output:

```
[12 13 14 15 16 10 12 10 95 17]
```

Join 2D arrays along rows or columns

```
mat1 = np.array([[1, 2, 4],
                 [3, 4, 5],
                 [7, 8, 9]])

mat2 = np.array([[1, 2, 5],
                 [4, 4, 9],
                 [6, 1, 9]])

print(np.concatenate((mat1, mat2), axis=0).shape)
print(np.concatenate((mat1, mat2), axis=1).shape)
```

Output:

```
(6, 3)
(3, 6)
```

Tricky points:

- `axis=0` joins along the first axis (rows in 2D).
- `axis=1` joins along the second axis (columns in 2D).
- All other dimensions must match.
- `np.concatenate((mat1 + mat2,))` concatenates only one array, so it simply returns that array's values. To join two matrices, pass both arrays.

Concatenate unequal-length 1D arrays

```
x = np.array([10, 20, 30, 40, 50])
y = np.array([10, 20, 30])
z = np.array([10, 20, 40, 50])

res = np.concatenate((x, y, z))

print(res)
print(res.shape)
```

Output:

```
[10 20 30 40 50 10 20 30 10 20 40 50]
(12,)
```

Important: 1D arrays of unequal lengths can be concatenated directly. If you reshape them into 2D rows, their column counts must match for concatenation along `axis=0`.

17. `np.stack()`

Definition: Combines arrays by adding a new axis.

```
arr = np.arange(1, 7).reshape(2, 3)
brr = np.arange(7, 13).reshape(2, 3)

ans = np.stack((arr, brr), axis=0)

print(ans)
print(ans.shape)
```

Output:

```
[[[ 1  2  3]
  [ 4  5  6]]

 [[ 7  8  9]
  [10 11 12]]]

(2, 2, 3)
```

Difference:

- `concatenate()` joins along an existing axis.
- `stack()` adds a new axis.
- Arrays passed to `stack()` must have the same shape.

18. `np.split()` and `np.array_split()`

Equal splitting

```
arr = np.arange(1, 13)
parts = np.split(arr, 3)

for part in parts:
    print(part)
```

Output:

```
[1 2 3 4]
[5 6 7 8]
[ 9 10 11 12]
```

`np.split()` requires an equal split when an integer number of sections is specified.

Unequal splitting

```
arr = np.arange(1, 18)
parts = np.array_split(arr, 4)

for part in parts:
    print(part, part.size)
```

Output:

```
[1 2 3 4 5] 5
[ 6  7  8  9] 4
[10 11 12 13] 4
[14 15 16 17] 4
```

Tricky point: `parts` is a Python list of arrays. `parts.size` is invalid. Use `part.size` inside the loop.

2D splitting

```
m = np.arange(1, 17).reshape(4, 4)

print(np.split(m, 2, axis=0))
print(np.split(m, 2, axis=1))
```

- `axis=0` : split along rows.
- `axis=1` : split along columns.
- `np.vsplit(m, 2)` : vertical split.
- `np.hsplit(m, 2)` : horizontal split.

`np.vsplit()` and `np.hsplit()` do not accept an `axis` argument.

Split a 3D array of shape `(2, 4, 4)`

```
x = np.arange(1, 17).reshape(4, 4)
y = np.arange(17, 33).reshape(4, 4)

d3 = np.array([x, y])
parts = np.split(d3, 2, axis=0)

print(d3.shape)
print(parts[0])
print(parts[1])
print(parts[0].shape, parts[1].shape)
```

Output:

```
(2, 4, 4)

[[[ 1  2  3  4]
  [ 5  6  7  8]
  [ 9 10 11 12]
  [13 14 15 16]]]

[[[17 18 19 20]
  [21 22 23 24]
  [25 26 27 28]
  [29 30 31 32]]]

(1, 4, 4) (1, 4, 4)
```

Each part retains the dimension along which it was split, so each output shape is `(1, 4, 4)`.

PART 5 — NUMERICAL ANALYSIS

19. Vectorized Operations

Definition: Apply operations to entire arrays without explicitly writing a Python loop for each element.

```
a = np.array([1, 2, 3, 4])

print(a + 5)
print(a * 2)
print(a ** 2)
```

Output:

```
[6 7 8 9]
[2 4 6 8]
[ 1  4  9 16]
```

Vectorization is useful because NumPy performs many operations in optimized compiled code.

20. Broadcasting

Definition: NumPy automatically expands compatible array dimensions for element-wise operations.

```
a = np.array([[1, 2, 3],
              [4, 5, 6]])

b = np.array([10, 20, 30])

print(a + b)
```

Output:

```
[[11 22 33]
 [14 25 36]]
```

Broadcasting rule: Compare dimensions from right to left. Each pair must be equal, or one of the dimensions must be 1.

- `(2, 3)` and `(3,)` are compatible.
- `(2, 3)` and `(2,)` are generally incompatible.

21. Aggregation Functions

Definition: Aggregations summarize array data into a single value or a smaller array.

```
a = np.array([10, 20, 30, 40, 50])

print(np.sum(a))
print(np.mean(a))
print(np.median(a))
print(np.min(a))
print(np.max(a))
print(np.std(a))
print(np.var(a))
print(np.argmax(a))
print(np.argmin(a))
```

Output:

```
150
30.0
30.0
10
50
14.142135623730951
200.0
4
0
```

Important functions:

- `sum()` — total

- `sum()` — total
- `mean()` — average
- `median()` — middle value
- `std()` — standard deviation
- `var()` — variance
- `argmax()` — index of the maximum
- `argmin()` — index of the minimum

For `np.std()` and `np.var()`, the default is population standard deviation and variance (`ddof=0`).

Find maximum value and its index

```
arr = np.array([10, 20, 30, 40, 50])

print(np.max(arr))
print(np.argmax(arr))
```

Output:

```
50
4
```

Tricky point: `np.argmax()` returns an index, not the maximum value. If multiple elements share the maximum, it returns the first matching index. For multidimensional arrays, flattening is the default unless `axis` is specified.

Axis-wise calculations

```
m = np.array([[1, 2, 3],
              [4, 5, 6]])

print(np.sum(m, axis=0))
print(np.sum(m, axis=1))
```

Output:

```
[5 7 9]
[ 6 15]
```

- `axis=0`: combine rows to get a result for each column.
- `axis=1`: combine columns to get a result for each row.

22. Normalization

Definition: Min-max normalization scales numerical values to a range, usually 0 to 1.

Formula:

`(x - minimum) / (maximum - minimum)`

```
arr = np.array([10, 20, 30, 40, 50])

normalized = (arr - arr.min()) / (arr.max() - arr.min())

print("Original:", arr)
print("Subtract:", arr - arr.min())
print("Normalized:", normalized)
```

Output:

```
Original: [10 20 30 40 50]
Subtract: [ 0 10 20 30 40]
Normalized: [0.   0.25 0.5  0.75 1.  ]
```

Tricky point: If all values are equal, `max - min` is zero. Handle this case before dividing.

23. Standardization (Z-score)

Definition: Standardization subtracts the mean and divides by the standard deviation.

```
arr = np.array([10, 20, 30, 40, 50])

standardized = (arr - arr.mean()) / arr.std()

print(standardized)
```

Output:

```
[-1.41421356 -0.70710678  0.          0.70710678  1.41421356]
```

Difference:

- Normalization usually maps values into 0–1.
- Standardization gives data a mean of 0 and standard deviation of 1 when standard deviation is nonzero.

24. Modulus / Remainder

```
x = np.array([10, 20, 30, 40, 50])
y = np.array([10, 20, 50, 30, 7])

print(x % y)
```

Output:

```
[0 0 30 10 1]
```

`%` calculates element-wise remainders. The arrays must be broadcast-compatible, and divisors must not be zero.

25. Sorting and Unique Values

```
                ([20, 10, 20, 10, 10])
```

```
arr = np.array([30, 10, 20, 10, 40])

print(np.sort(arr))
print(np.argsort(arr))
print(np.unique(arr))
print(np.unique(arr, return_counts=True))
```

Output:

```
[10 10 20 30 40]
[1 3 2 0 4]
[10 20 30 40]
(array([10, 20, 30, 40]), array([2, 1, 1, 1]))
```

- `np.sort()` returns sorted values.
- `np.argsort()` returns indices that sort the array.
- `np.unique()` returns unique values.
- `return_counts=True` also returns counts.

`np.sort(arr)` leaves the original array unchanged; `arr.sort()` sorts it in place.

26. Missing Values (NaN)

Definition: `np.nan` represents a missing numerical value in many NumPy workflows.

```
arr = np.array([10.0, 20.0, np.nan, 40.0])

print(np.isnan(arr))
print(np.isnan(arr).sum())
print(np.nanmean(arr))
print(np.nansum(arr))
```

Output:

```
[False False  True False]
1
23.333333333333332
70.0
```

Important:

- `np.isnan()` detects NaN values.
- `np.nanmean()` ignores NaNs while calculating the mean.
- `np.nansum()` ignores NaNs while calculating the sum.
- Ordinary `np.mean()` can return NaN if the array contains NaN.

PART 6 — MEMORY AND LINEAR ALGEBRA

27. Python List vs NumPy Array Memory

```
import sys
import numpy as np

values = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
arr = np.array(values)

print("List size:", sys.getsizeof(values))
print("Array size:", sys.getsizeof(arr))
print("Array data bytes:", arr.nbytes)
```

Important: `sys.getsizeof()` reports the shallow size of an object. A Python list's referenced element objects are not fully included in its reported size. `arr.nbytes` reports the array's element-buffer size, not all Python object overhead.

For large numerical datasets, NumPy often offers more compact storage and faster vectorized operations than Python lists.

28. Copy vs View

View: Shares underlying data when possible.

```
a = np.array([10, 20, 30, 40])
v = a[1:3]
v[0] = 99

print(a)
```

Output:

```
[10 99 30 40]
```

Copy: Independent data.

```
a = np.array([10, 20, 30, 40])
c = a.copy()
c[0] = 99

print(a)
print(c)
```

Output:

```
[10 20 30 40]
[99 20 30 40]
```

Tricky point: Basic slicing often returns a view; advanced indexing and `flatten()` generally return copies. Use `.copy()` when you need independent data.

29. Combining Arrays: `vstack()` and `hstack()`

```
```

```
a = np.array([1, 2])
b = np.array([3, 4])

print(np.vstack((a, b)))
print(np.hstack((a, b)))
```

Output:

```
[[1 2]
 [3 4]]

[1 2 3 4]
```

- `vstack()` stacks arrays vertically.
- `hstack()` joins arrays horizontally.
- Their shape requirements depend on the input dimensions.

30. Linear Algebra

Definition: Linear algebra deals with vectors, matrices and operations used in many Data Science and Machine Learning algorithms.

```
a = np.array([[1, 2],
              [3, 4]])

b = np.array([[5, 6],
              [7, 8]])

print(a @ b)
print(np.linalg.det(a))
print(np.linalg.inv(a))
```

Output:

```
[[19 22]
 [43 50]]

-2.000000000000004

[[-2.   1. ]
 [ 1.5 -0.5]]
```

Important:

- `a * b` is element-wise multiplication.
- `a @ b` is matrix multiplication.
- `np.linalg.det()` calculates a determinant.
- `np.linalg.inv()` calculates the inverse if the matrix is invertible.

Solve a system of equations

```
a = np.array([[2, 1]
```

```
a = np.array([[2, 1],
              [1, 3]])

b = np.array([5, 6])

print(np.linalg.solve(a, b))
```

Output:

```
[1.8 1.4]
```

This solves the matrix equation `Ax = b`.

PART 7 — ALL EXERCISES QUICK REVISION

Exercise 1: Multiples of 7 between 7 and 100

```
num = np.arange(7, 101, 7)

print(num)
print(len(num))
```

Output:

```
[ 7 14 21 28 35 42 49 56 63 70 77 84 91 98]
14
```

`np.array(range(7, 101, 7))` also works, but `np.arange()` is the direct NumPy approach.

Exercise 2: 4×4 array filled with 9

```
ans = np.full((4, 4), 9)
print(ans)
```

Output:

```
[[9 9 9 9]
 [9 9 9 9]
 [9 9 9 9]
 [9 9 9 9]]
```

Exercise 3: Convert integer array to float

```
a = np.array([1, 2, 3, 4, 5, 6, 7, 8, 9, 10])

print(a.dtype)

a = a.astype(float)

print(a)
print(a.dtype)
```

Output:

```
int64
[ 1.  2.  3.  4.  5.  6.  7.  8.  9. 10.]
float64
```

The precise integer and float dtype can vary by platform or explicit dtype choice.

Exercise 4: Compare Python list and NumPy array sizes

```
import sys

values = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
arr = np.array(values)

print(sys.getsizeof(values))
print(sys.getsizeof(arr))
print(arr.nbytes)
```

Note: The exact sizes depend on Python, NumPy, platform and dtype. Do not conclude the complete memory cost from only the list's shallow size.

Exercise 5: Four corners of a 6×6 matrix

```
arr = np.arange(1, 37).reshape(6, 6)

print(arr[0, 0], arr[0, -1],
      arr[-1, 0], arr[-1, -1])
```

Output:

```
1 6 31 36
```

Exercise 6: Every third element from index 1

```
arr = np.arange(1, 13)
print(arr[1::3])
```

Output:

```
[ 2  5  8 11]
```

Exercise 7: Reverse rows of a 5×5 array

```
arr = np.arange(1, 26).reshape(5, 5)
print(arr[::-1])
```

Output:

```
[[21 22 23 24 25]
 [16 17 18 19 20]
 [11 12 13 14 15]
 [ 6  7  8  9 10]
 [ 1  2  3  4  5]]
```

Exercise 8: Replace multiples of 3 with -1

```
arr = np.array([1, 3, 5, 6, 8, 9, 10, 12])

arr[arr % 3 == 0] = -1

print(arr)
```

Output:

```
[ 1 -1  5 -1  8 -1 10 -1]
```

Exercise 9: Normalize between 0 and 1

```
arr = np.array([10, 20, 30, 40, 50])

ans = (arr - arr.min()) / (arr.max() - arr.min())

print(ans)
```

Output:

```
[0.   0.25 0.5  0.75 1.  ]
```

Exercise 10: Transpose twice

```
arr = np.arange(1, 10).reshape(3, 3)

check = arr.T
again = check.T

print(again)
print(np.array_equal(arr, again))
```

Output:

```
[[1 2 3]
 [4 5 6]
 [7 8 9]]
True
```

Exercise 11: Maximum value and its index

```
arr = np.array([10, 20, 30, 40, 50])

print(np.max(arr))
print(np.argmax(arr))
```

Output:

```
50
```

```
50
4
```

The index is zero-based, so 50 is at index 4.

Exercise 12: Element-wise remainder

```
x = np.array([10, 20, 30, 40, 50])
y = np.array([10, 20, 50, 30, 7])

print(x % y)
```

Output:

```
[0 0 30 10 1]
```

Exercise 13: Stack two 2×3 arrays into shape (2, 2, 3)

```
arr = np.arange(1, 7).reshape(2, 3)
brr = np.arange(7, 13).reshape(2, 3)

ans = np.stack((arr, brr), axis=0)

print(ans)
print(ans.shape)
```

Output:

```
[[[ 1  2  3]
  [ 4  5  6]]

 [[ 7  8  9]
  [10 11 12]]]

(2, 2, 3)
```

Exercise 14: Concatenate three unequal-length 1D arrays

```
x = np.array([10, 20, 30, 40, 50])
y = np.array([10, 20, 30])
z = np.array([10, 20, 40, 50])

res = np.concatenate((x, y, z))

print(res)
print(res.shape)
```

Output:

```
[10 20 30 40 50 10 20 30 10 20 40 50]
(12,)
```

Exercise 15: Reshape a 1D array into 4×4 and transpose

Exercise 15: Reshape a 1D array into 4×4 and transpose

```
arr = np.arange(1, 17).reshape(4, 4)

print(arr)
print(np.transpose(arr))
```

Output:

```
[[ 1  2  3  4]
 [ 5  6  7  8]
 [ 9 10 11 12]
 [13 14 15 16]]

[[ 1  5  9 13]
 [ 2  6 10 14]
 [ 3  7 11 15]
 [ 4  8 12 16]]
```

Exercise 16: Split a 3D array of shape (2, 4, 4)

```
x = np.arange(1, 17).reshape(4, 4)
y = np.arange(17, 33).reshape(4, 4)

d3 = np.array([x, y])

parts = np.split(d3, 2, axis=0)

print(d3.shape)
print(parts[0].shape)
print(parts[1].shape)
```

Output:

```
(2, 4, 4)
(1, 4, 4)
(1, 4, 4)
```

Exercise 17: Split 17 elements into 4 parts

```
arr = np.arange(1, 18)
parts = np.array_split(arr, 4)

for part in parts:
    print(part, part.size)
```

Output:

```
[1 2 3 4 5] 5
```

```
[1 2 3 4 5] 5
[ 6  7  8  9] 4
[10 11 12 13] 4
[14 15 16 17] 4
```

PANDAS — COMPLETE NOTES FOR DATA ANALYSIS & DATA SCIENCE

1. What is Pandas?

Pandas is a Python library used to **create, manipulate, clean, analyze, and transform structured data**.

Common uses:

- CSV and Excel files read karna
- Missing values handle karna
- Duplicate rows remove karna
- Data filter aur sort karna
- Statistical analysis karna
- Group-wise analysis karna
- Data ko Machine Learning ke liye prepare karna

```
import pandas as pd
import numpy as np
```

2. Main Pandas Data Structures

Series

A Series is a one-dimensional labeled array.

```
s = pd.Series([10, 20, 30, 40])
print(s)
```

Output:

```
0    10
1    20
2    30
3    40
dtype: int64
```

Custom index:

```
s = pd.Series(
```

```
    [85, 90, 78],
    index=["Amit", "Neha", "Rahul"]
)

print(s)
print(s["Neha"])
```

Output:

```
Amit     85
Neha     90
Rahul    78
dtype: int64

90
```

DataFrame

A DataFrame is a two-dimensional tabular data structure containing rows and columns.

```
data = {
    "Name": ["Amit", "Neha", "Rahul"],
    "Age": [20, 21, 22],
    "Marks": [85, 92, 78]
}

df = pd.DataFrame(data)
print(df)
```

Output:

```
    Name  Age  Marks
0   Amit   20     85
1   Neha   21     92
2  Rahul   22     78
```

Remember: Series = one-dimensional; DataFrame = two-dimensional.

3. Creating DataFrames

From a dictionary

```
df = pd.DataFrame({
    "Name": ["Amit", "Neha"],
    "Marks": [85, 92]
})
```

From a list of lists

```
df = pd.DataFrame(
```

```
    [["Amit", 85], ["Neha", 92]],
    columns=["Name", "Marks"]
)
```

From a NumPy array

```
arr = np.array([[1, 2], [3, 4]])

df = pd.DataFrame(
    arr,
    columns=["A", "B"]
)

print(df)
```

Output:

```
   A  B
0  1  2
1  3  4
```

4. Inspecting Data

```
df.head()          # First 5 rows
df.head(3)         # First 3 rows
df.tail()          # Last 5 rows
df.tail(2)         # Last 2 rows

df.shape           # (rows, columns)
df.columns         # Column names
df.index           # Index labels
df.dtypes          # Data type of each column
df.info()          # Structure, data types, non-null counts
df.describe()      # Statistical summary of numeric columns
df.describe(include="all")  # Summary of all column types

df.size            # Total number of cells
df.ndim            # Number of dimensions
df.empty           # True if DataFrame has no elements
```

Example:

```
print(df.shape)
print(df.size)
```

If the DataFrame contains 3 rows and 3 columns:

```
(3, 3)
9
```

Tricky point: `shape` returns a tuple `(rows, columns)`, while `size` returns the total number of

cells.

5. Selecting Columns

```
df["Name"]                # One column: Series
df[["Name"]]              # One column: DataFrame
df[["Name", "Marks"]]     # Multiple columns
```

Important difference:

```
type(df["Name"])       # pandas Series
type(df[["Name"]])     # pandas DataFrame
```

Use double brackets when you want the result to remain a DataFrame.

6. Selecting Rows: loc and iloc

loc — label-based selection

```
df.loc[0]
df.loc[0, "Name"]
df.loc[0:2, ["Name", "Marks"]]
```

iloc — integer-position-based selection

```
df.iloc[0]          # First row
df.iloc[0, 1]       # First row, second column
df.iloc[0:3, 0:2]   # First 3 rows, first 2 columns
```

Most important difference:

- `loc` uses index labels; label-based slices include the ending label.
- `iloc` uses integer positions; Python-style slices exclude the ending position.

Example:

```
df.loc[0:2]     # Includes labels 0, 1, 2
df.iloc[0:2]    # Includes positions 0, 1
```

If your DataFrame has custom index labels, `loc[0]` may not mean the first row.

7. Filtering Data

Suppose:

```
df = pd.DataFrame({
    "Name": ["Amit", "Neha", "Rahul", "Priya"],
    "Age": [20, 21, 22, 19],
    "Marks": [85, 92, 78, 95]
})
```

Single condition

```
df[df["Marks"] > 85]
```

```
df[df["Age"] >= 21]
df[df["Name"] == "Neha"]
```

Multiple conditions

```
df[(df["Marks"] > 80) & (df["Age"] >= 20)]
```

Here:

- `&` means AND.
- `|` means OR.
- `~` means NOT.

Example:

```
df[(df["Marks"] > 90) | (df["Age"] < 20)]
```

Common mistake: Use `&` and `|` for Pandas conditions, not Python's `and` and `or`. Put each condition inside parentheses.

isin()

```
df[df["Name"].isin(["Amit", "Neha"])]
```

between()

```
df[df["Marks"].between(80, 90)]
```

By default, `between()` includes both boundaries.

query()

```
df.query("Marks > 80 and Age >= 20")
```

Useful for readable filtering expressions.

8. Sorting Data

```
df.sort_values("Marks")
df.sort_values("Marks", ascending=False)
df.sort_values(["Age", "Marks"], ascending=[True, False])
df.sort_index()
```

Example:

```
print(df.sort_values("Marks", ascending=False))
```

This sorts students from highest marks to lowest.

Remember: `sort_values()` sorts by column values; `sort_index()` sorts by index labels.

To change the existing DataFrame directly:

```
df.sort_values("Marks", ascending=False, inplace=True)
```

Otherwise, assign the result:

```
df = df.sort_values("Marks", ascending=False)
```

9. Adding and Updating Columns

```
df["Passed"] = df["Marks"] >= 40
df["Bonus"] = 5
df["FinalMarks"] = df["Marks"] + df["Bonus"]
```

Update a value:

```
df.loc[0, "Marks"] = 90
```

Update values matching a condition:

```
df.loc[df["Marks"] < 40, "Marks"] = 40
```

Create a column using conditions:

```
df["Result"] = np.where(
    df["Marks"] >= 40,
    "Pass",
    "Fail"
)
```

For multiple grade categories:

```
conditions = [
    df["Marks"] >= 90,
    df["Marks"] >= 75,
    df["Marks"] >= 60,
    df["Marks"] >= 40
]

choices = ["A+", "A", "B", "C"]

df["Grade"] = np.select(
    conditions,
    choices,
    default="Fail"
)
```

`np.select()` checks conditions in order. The first matching condition wins, so put the highest threshold first.

10. Deleting Rows and Columns

```
df.drop(columns=["Age"])
df.drop(index=[0, 1])
```

To modify the existing DataFrame:

```
df.drop(columns=["Age"], inplace=True)
```

Drop a row using its index label:

```
df.drop(index=2)
```

Tricky point: `drop(index=2)` removes the row with label `2`, not necessarily the third row.

11. Statistical Operations

```
df["Marks"].sum()
df["Marks"].mean()
df["Marks"].median()
df["Marks"].mode()
df["Marks"].min()
df["Marks"].max()
df["Marks"].count()
df["Marks"].std()
df["Marks"].var()
df["Marks"].nunique()
df["Marks"].value_counts()
```

Example:

```
marks = pd.Series([70, 80, 90, 90])

print(marks.mean())       # 82.5
print(marks.median())     # 85.0
print(marks.max())        # 90
print(marks.count())      # 4
print(marks.nunique())    # 3
print(marks.value_counts())
```

`value_counts()` counts the occurrences of each unique value.

By default, many Pandas numeric operations skip missing values. `count()` counts non-missing entries, whereas `size` on a Series counts all entries.

12. GroupBy — Extremely Important

`groupby()` groups rows based on a column so that you can calculate group-wise statistics.

```
df = pd.DataFrame({
    "Department": ["BCA", "MCA", "BCA", "MCA"],
    "Marks": [80, 90, 70, 85]
})
```

Average marks by department

```
df.groupby("Department")["Marks"].mean()
```

Output:

```
Department
```

```
BCA    75.0
MCA    87.5
```

Multiple aggregations

```
df.groupby("Department")["Marks"].agg(
    ["mean", "max", "min", "count"]
)
```

Group by multiple columns

```
df.groupby(["Department", "Age"])["Marks"].mean()
```

Use this when you need analysis by combinations of categories.

Group counts

```
df.groupby("Department").size()
```

Difference:

- `groupby(...).size()` counts rows in each group, including rows with missing values in other columns.
- `groupby(...).count()` counts non-missing values in the selected column.

13. Missing Values — NaN Handling

Missing data is one of the most important Data Analysis topics.

```
df.isnull()
df.isnull().sum()
df.notnull()
df.isna().sum()
```

`isnull()` and `isna()` are equivalent.

Fill missing values

```
df["Name"] = df["Name"].fillna("Unknown")
df["Department"] = df["Department"].fillna("Not Assigned")
df["Age"] = df["Age"].fillna(df["Age"].mean())
df["Marks"] = df["Marks"].fillna(df["Marks"].mean())
```

You can also fill all missing values with one value:

```
df.fillna(0)
```

Important: For numeric columns, use a numeric replacement. Filling a numeric column with `"Unknown"` may convert it to an unsuitable data type.

Drop missing values

```
df.dropna()                          # Drop rows containing any NaN
```

```
df.dropna(axis=1)                    # Drop columns containing any NaN
df.dropna(how="all")                 # Drop rows where all values are NaN
df.dropna(subset=["Marks"])          # Drop rows with missing Marks
df.dropna(thresh=3)                  # Keep rows with at least 3 non-null values
```

`dropna()` returns a new DataFrame by default; assign it or use `inplace=True` if you want to modify the existing DataFrame.

Forward fill and backward fill

```
df.ffill()     # Fill using previous valid value
df.bfill()     # Fill using next valid value
```

These are useful for ordered data, such as time-series records. Sort the data appropriately before filling.

14. Duplicate Rows

```
df.duplicated()
df.duplicated().sum()
df.drop_duplicates()
```

Keep the first occurrence by default.

```
df.drop_duplicates(keep="first")
df.drop_duplicates(keep="last")
df.drop_duplicates(keep=False)
```

Check duplicates based on selected columns:

```
df.duplicated(subset=["Name", "RollNo"])
df.drop_duplicates(subset=["Name", "RollNo"])
```

Important: `drop_duplicates()` compares row values, not just index labels. Two rows with different values are not duplicates merely because their index labels look similar.

15. Handling Data Types

```
df.dtypes
df["Age"] = df["Age"].astype(int)
df["Marks"] = df["Marks"].astype(float)
```

Only convert to `int` when the column has no missing values and its values are suitable for integer conversion.

For potentially messy numeric strings:

```
df["Marks"] = pd.to_numeric(
    df["Marks"],
    errors="coerce"
)
```

`errors="coerce"` converts invalid values to `NaN`.

For dates:

```
df["Date"] = pd.to_datetime(
    df["Date"],
    errors="coerce"
)
```

For text:

```
df["Name"] = df["Name"].astype("string")
```

16. String Operations

Pandas provides vectorized string methods through `.str`.

```
df["Name"].str.lower()
df["Name"].str.upper()
df["Name"].str.title()
df["Name"].str.strip()
df["Name"].str.contains("am", case=False, na=False)
df["Name"].str.replace("old", "new")
df["Name"].str.startswith("A", na=False)
df["Name"].str.len()
```

Example:

```
df["Name"] = df["Name"].str.strip()
```

This removes leading and trailing whitespace.

Tricky point: `.str` methods are useful for text columns. Handle missing values explicitly where needed, especially with filtering operations.

17. Renaming Columns and Resetting Index

```
df.rename(columns={"Marks": "Score"})
df.rename(columns={"Marks": "Score"}, inplace=True)
```

Reset index:

```
df.reset_index()
df.reset_index(drop=True, inplace=True)
```

If you want to discard the old index instead of adding it as a column, use `drop=True`.

Set a column as index:

```
df.set_index("RollNo")
```

18. Combining DataFrames

concat()

Combine DataFrames vertically:

```
result = pd.concat([df1, df2], axis=0)
```

Combine columns horizontally:

```
result = pd.concat([df1, df2], axis=1)
```

By default, `concat()` aligns columns or indexes by their labels. Missing matches can produce `NaN`.

Reset the combined index if needed:

```
result = pd.concat(
    [df1, df2],
    ignore_index=True
)
```

merge()

`merge()` combines data based on matching key columns, similar to a database join.

```
result = pd.merge(
    students,
    marks,
    on="RollNo",
    how="inner"
)
```

Common join types:

- `inner`: matching keys only
- `left`: all keys from the left DataFrame
- `right`: all keys from the right DataFrame
- `outer`: all keys from both DataFrames

Example:

```
result = pd.merge(
    students,
    marks,
    on="RollNo",
    how="left"
)
```

This preserves every student even if their marks record is missing.

join()

`join()` commonly combines DataFrames using their indexes.

Interview difference: `concat()` stacks/alines objects, `merge()` joins using keys, and `join()` commonly joins on indexes.

19. Pivot Tables and Crosstabs

pivot_table()

Summarize numeric values by categories.

```
pd.pivot_table(
```

```
    df,
    values="Marks",
    index="Department",
    columns="Gender",
    aggfunc="mean"
)
```

This calculates average marks for each Department-Gender combination.

crosstab()

Count combinations of categories:

```
pd.crosstab(
    df["Department"],
    df["Gender"]
)
```

Use `pivot_table()` for numeric aggregation and `crosstab()` for frequency tables.

20. Ranking and Top N

```
df["Rank"] = df["Marks"].rank(
    ascending=False,
    method="dense"
)
```

Get top students:

```
df.nlargest(3, "Marks")
```

Get bottom students:

```
df.nsmallest(3, "Marks")
```

Ranking methods:

- `average` : average rank for ties
- `min` : lowest rank for ties
- `max` : highest rank for ties
- `first` : ranks ties in order of appearance
- `dense` : ties share rank without gaps

21. Apply, Map and Vectorization

map() — usually for a Series

```
df["Department"].map({
    "BCA": "Computer Applications",
    "MCA": "Computer Applications Masters"
})
```

Values not present in the mapping dictionary become missing values.

apply()

```
df["Marks"].apply(lambda x: x + 5)
```

Apply a function to each value in a Series.

For a DataFrame:

```
df[["Age", "Marks"]].apply(
    lambda col: col.max()
)
```

This calculates the maximum of each selected column.

Vectorized operations

```
df["Marks"] + 5
df["Marks"] * 2
df["Marks"] >= 40
```

Prefer vectorized Pandas operations over Python loops for common column calculations. `apply()` is useful when a simple vectorized operation is not enough.

22. Reading and Writing Files

CSV

```
df = pd.read_csv("students.csv")
df.to_csv("output.csv", index=False)
```

Excel

```
df = pd.read_excel("students.xlsx")
df.to_excel("output.xlsx", index=False)
```

Reading Excel files generally requires an appropriate Excel engine, such as `openpyxl` for `.xlsx` files.

Useful reading options

```
df = pd.read_csv(
    "students.csv",
    usecols=["Name", "Marks"],
    nrows=100
)
```

Other useful options:

```
pd.read_csv("students.csv", na_values=["NA", "missing"])
pd.read_csv("students.csv", skiprows=1)
```

23. Time-Series Data

```
df["Date"] = pd.to_datetime(df["Date"])
```

```
df = df.sort_values("Date")
df = df.set_index("Date")
```

Extract date parts:

```
df.index.year
df.index.month
df.index.day
df.index.dayofweek
```

Resample time-series data:

```
df.resample("ME")["Sales"].sum()
```

`"ME"` represents month-end frequency in current Pandas versions. Frequency aliases can vary between older and newer Pandas versions.

24. Correlation and Data Analysis

```
df.corr(numeric_only=True)
```

Correlation measures how numerical variables vary together.

- Close to `+1` : strong positive linear relationship
- Close to `-1` : strong negative linear relationship
- Close to `0` : little or no linear relationship

Important: Correlation does not prove causation. A correlation matrix may also be misleading when the data contains outliers or other confounding factors.

25. A Complete Data Cleaning Workflow

```
import pandas as pd
```

```
import numpy as np

df = pd.read_csv("students.csv")

# 1. Inspect data
print(df.head())
print(df.shape)
print(df.info())
print(df.isnull().sum())

# 2. Remove duplicate rows
df = df.drop_duplicates()

# 3. Clean text columns
df["Name"] = df["Name"].astype("string").str.strip()

# 4. Convert numeric data safely
df["Marks"] = pd.to_numeric(
    df["Marks"],
    errors="coerce"
)

# 5. Handle missing values
df["Name"] = df["Name"].fillna("Unknown")
df["Marks"] = df["Marks"].fillna(df["Marks"].median())

# 6. Create a derived column
df["Passed"] = df["Marks"] >= 40

# 7. Sort data
df = df.sort_values("Marks", ascending=False)

# 8. Save cleaned data
df.to_csv("cleaned_students.csv", index=False)
```

Real-world rule: Inspect data before cleaning. Choose whether to fill or remove missing values based on the column's meaning and the analysis objective.

26. Your Practiced Exercise — Missing Values and Duplicates

```
import pandas as pd
```

```
import numpy as np

data = {
    "RollNo": [101, 102, 103, 104, 105, 102, 107, 108],
    "Name": ["Amit", "Neha", "Rahul", "Priya", "Rohan", "Neha", "Arjun", np.nan],
    "Department": ["BCA", "BCA", "MCA", np.nan, "MCA", "BCA", "BCA", "MCA"],
    "Age": [20, np.nan, 22, 20, np.nan, np.nan, 21, 24],
    "Marks": [85, 92, np.nan, 78, 88, 92, np.nan, 73]
}

df = pd.DataFrame(data)

print(df)
```

Exercise A: Count missing values

```
print(df.isnull().sum())
```

Expected output:

```
RollNo        0
Name          1
Department    1
Age           3
Marks         2
dtype: int64
```

Exercise B: Fill missing values

```
df["Name"] = df["Name"].fillna("Unknown")
df["Department"] = df["Department"].fillna("Not Assigned")
df["Age"] = df["Age"].fillna(df["Age"].mean())
df["Marks"] = df["Marks"].fillna(df["Marks"].mean())
```

Exercise C: Remove duplicate rows

```
print(df.duplicated().sum())
df = df.drop_duplicates()
```

Before filling missing values, the two Neha rows are exact duplicates. After filling, the same values are still present in both rows, so they remain duplicates.

Exercise D: Drop rows with missing Marks

```
df.dropna(subset=["Marks"])
```

Do this before filling Marks if your objective is to remove rows with missing marks. If you have already filled all missing Marks values, this operation will not remove those rows.

Exercise E: Department-wise average marks

```
df.groupby("Department")["Marks"].mean()
```

Exercise F: Highest marks

```
df["Marks"].max()
```

Exercise G: Students with marks above 80

```
df[df["Marks"] > 80]
```

Exercise H: Sort marks in descending order

```
df.sort_values("Marks", ascending=False)
```